The `mv::memvar` Library

A Complete Technical Guide to `memvar.h`

A source-level analysis of a header-only C++20 “variable with memory” facility

---

Covers: motivation, design, storage model, type constraints, public API, usage patterns, edge cases, and known limitations

---

Analyzed files: `memvar.h`, `is_string.h` - v.1.1.0 (C++20)

July 25, 2026

Contents

1 Introduction and Motivation **3**
- 1.1 What the library provides . . . 3
- 1.2 Why a "variable with memory" . . . 3
- 1.3 Structural relationship between the two files . . . 3

2 `is_string.h`: Type Trait and Diagnostics **5**
- 2.1 Overview . . . 5
- 2.2 `mv::type<T>()`: demangled type names . . . 5
- 2.3 `mv::is_string<T>` and `mv::is_string_v<T>` . . . 6

3 `memvar.h`: Architecture **8**
- 3.1 Header-level content . . . 8
- 3.2 Global `operator«` overloads for wide/Unicode strings . . . 8
- 3.3 `mv::is_bigint<T>`: an extensibility hook . . . 9
- 3.4 `IsAnyOf` and `AnyStandardString`: concepts . . . 9

4 `memvarBase`: the Non-Template Root **11**
- 4.1 Purpose and design . . . 11

5 `memvar<T>`: Core Class Template **13**
- 5.1 Class declaration and storage . . . 13
- 5.2 `checkType()`: the type-admissibility guard . . . 13
- 5.3 `getValue()` and `setValue()` . . . 14
- 5.4 Increment/decrement helpers and the string exclusion . . . 15
- 5.5 `memvarPrinter()`: shared history-printing engine . . . 15
- 5.6 Constructors and destructor . . . 16
- 5.7 Conversion, call, and subscript operators . . . 17
- 5.8 Assignment and compound-assignment operators . . . 17
- 5.9 Increment and decrement operators . . . 18
- 5.10 History introspection API . . . 18
- 5.11 Free (non-member) helper . . . 19
- 5.12 Free relational and arithmetic operators . . . 20

6 `memvarTimed<T, Time, Clock>`: Time-Tagged History **21**
- 6.1 Purpose . . . 21
- 6.2 Constructors . . . 22
- 6.3 Assignment operators and value accessors . . . 22
- 6.4 Time-tag accessors and printers . . . 23
- 6.5 `clearHistory()` override . . . 23
- 6.6 `getHistoryValue` override and `getHistoryTimedValue` . . . 24
- 6.7 Free `getHistoryValue` and `operator«` for `memvarTimed` . . . 24

7 Usage Examples **26**
7.1 Basic construction, assignment, and value access . . . 26
7.2 Explicit construction with a custom history capacity . . . 26
7.3 Invalid capacity: exception on construction . . . 27
7.4 String-valued `memvar` . . . 27
7.5 Wide and Unicode string `memvar`s and printing . . . 27
7.6 Prefix/postfix increment and decrement . . . 27
7.7 History iteration, printing, min/max . . . 27
7.8 Indexed access and out-of-range handling . . . 28
7.9 Free function `getHistoryValue` and `historyValue` tuple . . . 28
7.10 Clearing history . . . 28
7.11 Copy semantics: what is and is not allowed . . . 29
7.12 Comparing and combining `memvar` values . . . 29
7.13 Using `memvarTimed` . . . 29
7.14 Using `mv::type<T>()` and `mv::is_string_v<T>` directly . . . 30
7.15 A custom "bigint-like" extension type . . . 30

8 Edge Cases **32**
8.1 Type rejected only at first construction, not at first naming . . . 32
8.2 Multi-character separators in printers corrupt output . . . 32
8.3 Printing history to a non-terminal stream . . . 32
8.4 Division and the absence of divide-by-zero protection . . . 32
8.5 Postfix increment near the enforced minimum capacity . . . 33
8.6 `memvarTimed` with a non-default `Time`/`Clock` and the free helpers . . . 33
8.7 Slicing-free but dispatch-inconsistent access through base references . . . 33

9 Limitations, Constraints, and Open Issues **34**
9.1 Type-safety and validation . . . 34
9.2 The unused error flag . . . 34
9.3 Output/formatting correctness . . . 35
9.4 Concurrency . . . 35
9.5 Exception safety . . . 35
9.6 API consistency . . . 35
9.7 Extensibility and completeness . . . 36
9.8 Summary table of constraints . . . 37

10 Conclusion **38**

Chapter 1

Introduction and Motivation

1.1 What the library provides

The two headers under analysis, `is_string.h` and `memvar.h`, together implement a small, header-only C++20 utility library centered on one class template: `mv::memvar<T>`. A `memvar<T>` behaves, from the outside, like a plain scalar or string variable of type `T`: it can be constructed, assigned, compared, printed, incremented/decremented and used in arithmetic expressions. What distinguishes it from a plain `T` is that *every value it is ever assigned is retained*, up to a configurable capacity, in an internal history buffer. Reading the "current" value is simply reading the most recently pushed entry; older entries remain retrievable by index, can be iterated, printed, and reduced (minimum/maximum).

`is_string.h` is a small supporting header. It contributes two independent pieces of functionality that `memvar.h` depends on:

- `mv::is_string<T>` / `mv::is_string_v<T>` — a trait that recognizes the five standard string specializations (`std::string`, `std::wstring`, `std::u8string`, `std::u16string`, `std::u32string`).
- `mv::type<T>()` — a helper returning a human-readable (demangled, on GCC/Clang) name for a type, used purely as a diagnostic/introspection utility.

1.2 Why a "variable with memory"

Reading the class purely from its interface, `memvar<T>` is designed for scenarios where a program needs not just the current value of a quantity but also a short rolling history of how that quantity changed: for example, a control loop that needs to compare the last few readings of a sensor, a UI widget that wants to support "undo" to a bounded depth, or logging/instrumentation code that wants a compact record of the last N writes to a variable without introducing an external logging subsystem. The type restricts itself, via a compile-time-triggered check, to integral types, floating-point types, the five standard string types, and one explicitly whitelisted external type, `bip::bigint`, that is only forward-declared in the header.

1.3 Structural relationship between the two files

`memvar.h` `#includes` `is_string.h` and reuses `is_string_v<T>` inside its own type-admissibility check (`memvar<T>::checkType()`) and inside a C++20 `concept` (`AnyStandardString`) that is later used to selectively enable or disable arithmetic and increment/decrement operators depending on whether `T` is string-like. `is_string.h` itself has no dependency on `memvar.h`; it is a self-contained, reusable trait/utility header.

File	Role
`is_string.h`	Type trait (`is_string`) + RTTI name demangling utility (`type<T>()`)
`memvar.h`	`memvarBase`, `memvar<T>`, `memvarTimed<T,Time,Clock>`, free operators

Chapter 2

`is_string.h`: Type Trait and Diagnostics

2.1 Overview

`is_string.h` defines everything inside namespace `mv` and has no other library dependency besides the standard library (`<cstdlib>`, `<cstring>`, `<string>`, `<typeid>`/`<typeinfo>`, `<type_traits>`, `<memory>`, and, conditionally, `<cxxabi.h>`).

2.2 `mv::type<T>()`: demangled type names

The header defines a function template `mv::type<T>()` returning a `std::string`. Its implementation is conditionally compiled based on whether the macro `NO_DEMANGLE` is defined *before* the header is included:

```
#ifndef NO_DEMANGLE
#include <cxxabi.h>
namespace mv {
template <typename T>
std::string type () {
  static_assert(sizeof(T) > 0, "T must be complete");
  auto tn {typeid(T).name()};
  int status {0};
  std::unique_ptr<char, decltype(&free)> demangled(
      abi::__cxa_demangle(tn, nullptr, nullptr, &status), free);
  std::string result = (status == 0 && demangled) ? demangled.get() : tn;
  return result;
}
}
#else
namespace mv {
template <typename T>
constexpr std::string type () {
  static_assert(sizeof(T) > 0, "T must be complete");
  return typeid(T).name();
}
}
#endif
```

Listing 2.1: Two mutually exclusive implementations of `mv::type<T>()`

Behavior

- When `NO_DEMANGLE` is *not* defined (the default, since it is never defined by the header itself), `mv::type<T>()` calls `typeid(T).name()` to obtain the implementation-defined mangled name, then passes it through the Itanium C++ ABI demangler `abi::__cxa_demangle`. The demangled buffer, a raw `char*` allocated with `malloc` by the ABI library, is wrapped in a `std::unique_ptr<char, decltype(&free)>` so it is released with `free` rather than `delete`, matching the allocator used internally by `__cxa_demangle`. If demangling fails (`status != 0`) or the pointer is null, the function falls back to the raw mangled name.
- When `NO_DEMANGLE` *is* defined, the fallback implementation simply returns the raw, non-demangled name from `typeid(T).name()`. It is declared `constexpr`, although in practice `typeid(T).name()` is not usable in a constant expression for arbitrary polymorphic or non-trivial types on most implementations, so the practical benefit of the `constexpr` qualifier is limited to permitting (not guaranteeing) compile-time evaluation; the function will typically still run at ordinary runtime.
- Both variants perform a `static_assert(sizeof(T) > 0, ...)`, which forces `T` to be a complete type at the point of instantiation. This is the only compile-time guard in the header; it rejects only incomplete types, not any other property of `T`.
- The comment-free code shows the header explicitly limits itself, by its own conditional compilation guard, to GCC/Clang-style Itanium ABI name mangling; no branch exists for MSVC's `name()` format or demangling API.

Design Note

The two overloads are not simultaneously visible: the preprocessor choice is a source-level switch. A translation unit that defines `NO_DEMANGLE` before including the header gets the non-demangling, `constexpr`-qualified overload; one that does not gets the demangling, non-`constexpr` overload. Because this is decided per translation unit, mixing translation units with and without `NO_DEMANGLE` in the same binary is possible without a compile error (both instantiate to a function of the same signature, `std::string mv::type<T>()`), but this violates the One Definition Rule (ODR) if the same specialization `mv::type<T>()` for a given `T` is instantiated in more than one translation unit with different bodies. This is a latent ODR hazard inherent in guarding a function template's *body* (rather than only auxiliary code) with a translation-unit-local macro.

2.3 `mv::is_string<T>` and `mv::is_string_v<T>`

```
namespace mv {
template <typename T>
struct is_string {
    using U = std::remove_cvref_t<T>;
    static constexpr bool value =
        std::is_same_v<U, std::string> ||
        std::is_same_v<U, std::wstring> ||
        std::is_same_v<U, std::u8string> ||
        std::is_same_v<U, std::u16string> ||
        std::is_same_v<U, std::u32string>;
};
template <typename T>
constexpr bool is_string_v = is_string<T>::value;
}
```

Listing 2.2: The `is_string` trait

Behavior

- `is_string<T>` first strips cv-qualification and reference-ness from `T` via `std::remove_cvref_t` (a C++20 trait), producing `U`. It then checks `U` against exactly the five standard specializations of `std::basic_string` that have named aliases in the standard library: `std::string`, `std::wstring`, `std::u8string`, `std::u16string`, `std::u32string`.
- Because `remove_cvref_t` is applied, `is_string_v<const std::string&>`, `is_string_v<std::string&&>`, and `is_string_v<const std::string>` are all `true`.
- `std::string_view` and any custom `std::basic_string<CharT, Traits, Alloc>` instantiation with a non-standard `CharT`/`Traits`/`Alloc` (e.g. `std::basic_string<char, MyTraits>`) is *not* recognized: the trait tests exact type identity against the five aliases, not "is a `std::basic_string` instantiation".
- The trait performs no SFINAE and cannot fail to instantiate; `is_string_v<T>` is well-formed for every complete or incomplete non-function, non-array type `T` (the `std::is_same_v` checks do not require completeness).

Chapter 3

memvar.h: Architecture

3.1 Header-level content

Besides the class templates, `memvar.h` contributes, at global/namespace scope:

- Four `operator«` overloads for `std::wstring`, `std::u8string`, `std::u16string`, `std::u32string` against `std::ostream`, defined at global scope *outside* namespace `mv`, before the `mv` namespace block even opens.
- A forward declaration `namespace bip { class bigint; }`.
- Inside namespace `mv`: `is_bigint<T>` / `is_bigint_v<T>`, the `IsAnyOf` and `AnyStandardString` concepts, `memvarBase`, `memvar<T>`, and (later, after the free operators) `memvarTimed<T, Time, Clock>`.
- At global scope, outside namespace `mv`: `operator«` for `memvar<T>`, and the full set of relational (`==`, `>`, `<`, `>=`, `<=`) and arithmetic (`+`, `-`, `*`, `/`) free operator templates, each with three overloads (memvar/memvar, memvar/T, T/memvar).
- Finally, `operator«` for `memvarTimed<T>`.

3.2 Global `operator«` overloads for wide/Unicode strings

```
inline std::ostream& operator<<(std::ostream& os, const std::wstring& s) {
  std::wstring_convert<std::codecvt_utf8<wchar_t>> conv;
  return os << conv.to_bytes(s);
}
inline std::ostream& operator<<(std::ostream& os, const std::u8string& s) {
  return os << reinterpret_cast<const char*>(s.c_str());
}
inline std::ostream& operator<<(std::ostream& os, const std::u16string& s) {
  std::wstring_convert<std::codecvt_utf8_utf16<char16_t>, char16_t> conv;
  return os << conv.to_bytes(s);
}
inline std::ostream& operator<<(std::ostream& os, const std::u32string& s) {
  std::wstring_convert<std::codecvt_utf8<char32_t>, char32_t> conv;
  return os << conv.to_bytes(s);
}
```

Listing 3.1: Stream insertion for non-`char` string types

These four overloads exist because `std::ostream::operator«` has no built-in overload for `std::wstring`, `std::u8string`, `std::u16string`, or `std::u32string` (a narrow-character stream can only directly stream `char`-based sequences). They convert each wide/Unicode representation into a

UTF-8 encoded `std::string` (or, for `std::u8string`, reinterpret the bytes directly, since `char8_t` and `char` share representation for valid UTF-8 data), then insert that into the stream.

- The `std::wstring` overload uses `std::codecvt_utf8<wchar_t>`, which treats the wide character as a UTF-32 code point. This is correct on platforms where `wchar_t` is 32 bits (e.g. Linux/glibc, most Unix-like systems), but is *not* correct on platforms where `wchar_t` is 16 bits (notably Windows, where `wchar_t` encodes UTF-16 and surrogate pairs would need `codecvt_utf8_utf16<wchar_t>` instead, as is used for `char16_t` two lines below). This makes the `std::wstring` overload platform-dependent in correctness.
- The `std::u8string` overload performs a bare `reinterpret_cast<const char*>(s.c_str())`. This assumes (a) that `s` contains no embedded NUL bytes before its logical end (since it depends on `c_str()`'s NUL terminator, discarding `s`'s stored `size()`), and (b) that whatever consumes the byte stream from `os` interprets it as UTF-8 (true for most modern terminals/files, not guaranteed in general, e.g. legacy Windows consoles).
- All four overloads rely on `std::wstring_convert`/`std::codecvt_utf8*`, which were deprecated in the C++ standard as of C++17. The header uses them anyway because, as of C++20, the standard has not replaced them with a straightforward alternative; this is a source of deprecation warnings and a forward-compatibility risk against future standard library versions that may remove these facilities.
- These operators are declared `inline` and are defined at global (not `mv`) namespace scope. Because `std::wstring` etc. are library types, this is technically adding overloads for standard library types from a user header at global scope; found via ordinary lookup at the call site rather than ADL (since neither operand type lives in a user namespace), so any call site must have this header (or another declaration with identical signature) visible via `#include` or transitively.

3.3 `mv::is_bigint<T>`: an extensibility hook

```
namespace bip { class bigint; }
namespace mv {
template <typename T>
struct is_bigint : std::false_type {};
template <>
struct is_bigint<bip::bigint> : std::true_type {};
template <typename T>
inline constexpr bool is_bigint_v = is_bigint<T>::value;
}
```

The primary template says "no type is a bigint" by default; a single full specialization opts `bip::bigint` in. `bip::bigint` is only *forward*-declared in this header (its definition presumably lives in a separate `bigint.h`, referenced only in the file's comments as being used by unit tests, but not included here). A full template specialization does not require `bip::bigint` to be a complete type — only its declaration is needed to name it as a specialization argument — so this compiles even without `bigint.h`. This is the library's designated *extension point*: to allow another custom numeric type `X` to be stored in a `memvar<X>`, a consumer would add an analogous `template<> struct mv::is_bigint<X> : std::true_type {};` specialization (this is exactly how `checkType()`, described below, is written to accept it).

3.4 `IsAnyOf` and `AnyStandardString`: concepts

```
template <typename T, typename... Types>
```

```
concept IsAnyOf = (std::same_as<T, Types> || ...);
template <typename T>
concept AnyStandardString = IsAnyOf<std::remove_cvref_t<T>,
                                    std::string, std::wstring,
                                    std::u8string, std::u16string, std::u32string>;
```

`IsAnyOf` is a generic, reusable C++20 concept: `IsAnyOf<T, A, B, C>` is satisfied iff `T` is exactly one of `A`, `B`, `C` (fold expression over `std::same_as`). `AnyStandardString<T>` instantiates `IsAnyOf` with the same five standard string types tested by `mv::is_string`, after stripping cv/reference qualification.

Design Note

`AnyStandardString` is functionally redundant with `mv::is_string_v`: both encode the same five-type list. The codebase does not express one in terms of the other (e.g. `AnyStandardString` does not call `is_string_v`), so the list of five string types is duplicated verbatim in two places (`is_string.h`'s `is_string` and `memvar.h`'s `AnyStandardString`). Any future change to the set of recognized string types (e.g. adding a custom string class) must be made in both places to stay consistent — a maintenance hazard inherent in the current source, independent of any comment describing intent.

Chapter 4

memvarBase: the Non-Template Root

```cpp
class memvarBase {
public:
  using capacityType = int64_t;
  memvarBase(const memvarBase& rhs) = delete;
  memvarBase& operator=(const memvarBase& rhs) = delete;
  memvarBase(memvarBase&& rhs) = delete;
  memvarBase& operator=(memvarBase&& rhs) = delete;
  capacityType getHistoryCapacity() const noexcept { return historyCapacity_; }
protected:
  static constexpr capacityType minimumHistoryCapacity_ {2};
  static constexpr capacityType historyCapacityDefault_ {10};
  const capacityType historyCapacity_ {historyCapacityDefault_};
  explicit memvarBase(capacityType historyCapacity) noexcept : historyCapacity_ (
      historyCapacity) {}
  memvarBase() = default;
  virtual ~memvarBase() = default;
};
```

Listing 4.1: `memvarBase`

4.1 Purpose and design

`memvarBase` is a small, non-template base class whose reason for existing is to give `memvar<T>` (for every `T`) a common, type-erased base type, and to hold the *capacity* concept independently of the stored value type `T`. This lets code refer to "some memvar, regardless of `T`" through a `memvarBase&`/`memvarBase*`, at least for capacity-related queries (`getHistoryCapacity()`), even though the header never exercises this polymorphically beyond `memvarTimed` overriding virtual members declared in `memvar<T>`.

Key facts

- Copy and move construction/assignment are all explicitly `= deleted`. Every `memvar<T>` object is therefore non-copyable and non-movable *at the base level*; this propagates to every derived class unless the derived class supplies its own special member functions that do not rely on the (deleted) base ones for those operations (see the discussion of `memvar<T>`'s own special members below — notably, its `operator=(const memvar&)` is user-defined, not defaulted, and thus does not require the deleted base copy-assignment).
- Both constructors (`memvarBase()` and `explicit memvarBase(capacityType)`) are `protected`, and the class has a `virtual` destructor but no pure virtual functions. This makes `memvarBase`

effectively non-instantiable directly from outside a derived class (protected constructors), even though it is not formally abstract in the C++ sense (there is no pure virtual function).

- `capacityType` is a public type alias to `int64_t` — a fixed-width, signed 64-bit integer, used throughout the library for history sizes and indices.
- `minimumHistoryCapacity_` (2) and `historyCapacityDefault_` (10) are `protected static constexpr` constants, visible to `memvar<T>` but not to outside callers.
- `historyCapacity_` is a `const` data member, defaulted (via a default member initializer) to `historyCapacityDefault_` (10), or set explicitly through the `explicit memvarBase(capacityType historyCapacity)` constructor. Because it is `const`, once a `memvarBase` subobject is constructed its capacity can never be changed for the lifetime of the object — there is no setter.
- The one-argument constructor is `noexcept` and performs *no validation* of the `historyCapacity` argument (it does not check it against `minimumHistoryCapacity_`); that check, when it happens at all, is performed later, by `memvar<T>`'s own constructor (see §5.6), not by `memvarBase` itself.

Open Issue / Limitation

Validation asymmetry: only the `memvar(const T&, capacityType)` constructor of `memvar<T>` checks `historyCapacity_ < minimumHistoryCapacity_` and throws. The default `memvar()` constructor never passes a custom capacity (it always uses the `memvarBase` default constructor, which yields the always-valid default of 10), so it never needs the check — but this means the *only* code path that can construct an invalid-capacity object is guarded, while `memvarBase(capacityType)` itself, callable directly by any future derived class other than `memvar<T>`, performs no such check and is `noexcept`, so a new derived class that forwards an out-of-range capacity to `memvarBase` without repeating `memvar`'s validation would silently accept it.

Chapter 5

memvar<T>: Core Class Template

5.1 Class declaration and storage

```
template <typename T>
class memvar : public memvarBase {
protected:
  using memvarHistory = std::deque<T>;
  memvarHistory memo_ {};
  static void checkType() { ... }
  const T& getValue() const { return memo_.at(0); }
  virtual void setValue(const T& value) { ... }
  ...
public:
  using historyValue = std::tuple<T, bool>;
  ...
};
```

Storage model

The entire history of values is stored in a single `std::deque<T>` named `memo_`, declared `protected` (visible to `memvar<T>` itself and to derived classes such as `memvarTimed<T,...>`, but not to outside code). `std::deque` was chosen (over, e.g., `std::vector` or `std::list`) because the class needs efficient insertion at the *front* (`emplace_front`, for the newest value) and efficient removal at the *back* (`pop_back`, to drop the oldest value once capacity is exceeded); `deque` provides amortized O(1) for both operations, whereas a `vector` would need O(n) shifting for front insertion and a singly-linked `list` could not efficiently pop from the back without an extra tail pointer (it can, but `deque` also offers `O(1)` random-access indexing via `at()`, which the class also relies on – `std::list` does not).

Index convention: element `0` of `memo_` is always the *most recently set* value (the "current" value); higher indices are progressively older values. This follows directly from `setValue` always calling `memo_.emplace_front(value)`.

5.2 checkType(): the type-admissibility guard

```
static void checkType() {
    static_assert((std::is_integral_v<T> != false ||
                   std::is_floating_point_v<T> != false ||
                   is_string_v<T> != false ||
                   is_bigint_v<T> != false),
```

```
                "String, integral, floating point, or bigint types required.");
}
```

`checkType()` is a `static`, non-template member function of the class template `memvar<T>`. Its body contains a single `static_assert` that requires `T` to satisfy at least one of: `std::is_integral_v<T>`, `std::is_floating_point_v<T>`, `mv::is_string_v<T>`, or `mv::is_bigint_v<T>`.

Design Note

Because `checkType` is a member function of a class *template*, its body is only type-checked (and thus only instantiated, and thus only capable of firing the `static_assert`) when the function is actually *used* ("odr-used" or otherwise triggers implicit instantiation) for a specific `T`. Every `memvar<T>` constructor calls `checkType()` in its body, so in practice the assertion fires as soon as any object of a disallowed `memvar<BadType>` is constructed. However, merely *naming* the type `memvar<BadType>` (e.g. as a function parameter type, in a `using` alias, inside `sizeof(memvar<BadType>)`, or as a non-constructed data member type) does *not* trigger the check, because it does not instantiate the constructor body. This is a materially different, weaker guarantee than expressing the constraint as a `requires` clause on the class template itself (e.g. `template <typename T> requires (...) class memvar`), which would reject the type at the point the template is named with unsatisfying arguments, not merely at first construction. The current design defers the check to first-use.

5.3 `getValue()` and `setValue()`

```
const T& getValue() const { return memo_.at(0); }
virtual void setValue(const T& value) {
  memo_.emplace_front(value);
  if ( static_cast<capacityType>(memo_.size()) > historyCapacity_ ) {
    memo_.pop_back();
  }
}
```

- `getValue()` returns a `const T&` to element 0 of `memo_`, using `.at(0)` (bounds-checked, throws `std::out_of_range` if `memo_` is empty) rather than `[0]`. In every reachable state produced by the class's own public API, `memo_` always contains at least one element after construction (every constructor calls `memo_.emplace_front(...)` exactly once, and `clearHistory()` clears then immediately re-inserts a default value), so this bound check should not fail through ordinary use; it exists as a defensive measure, e.g. against a derived class that manipulates `memo_` directly and empties it without re-inserting.
- `setValue()` is declared `virtual`, and is the single choke point through which every mutating operation (assignment, compound assignment, increment/decrement) pushes a new value: it always `emplace_front`s the new value, then, if the resulting size exceeds `historyCapacity_`, pops exactly one element from the back. Because at most one element is ever added per call, at most one needs to be (and is) removed to restore the invariant `size() <= historyCapacity_`.
- `setValue` is `protected`; it is not directly callable by outside code, only through the class's public operators (`operator=`, `operator+=`, etc.) and `clearHistory()`.
- Being `virtual` is what allows `memvarTimed<T,...>` to override it and transparently attach a synchronized time-tag history (§6).

5.4 Increment/decrement helpers and the string exclusion

```
T incr1() requires (!AnyStandardString<T>) {
  const T newValue {static_cast<T>(getValue() + 1)};
  setValue(newValue);
  return newValue;
}
void voidIncr1() requires (!AnyStandardString<T>) { setValue(getValue() + 1); }
T decr1() requires (!AnyStandardString<T>) { ... }
void voidDecr1() requires (!AnyStandardString<T>) { ... }
```

These four `protected` helpers back the public `operator++`/`operator-` (prefix/postfix) pair. Each carries a `requires (!AnyStandardString<T>)` clause, so they participate in overload resolution/exist as valid members only when `T` is *not* one of the five standard string types (arithmetic `+ 1`/`- 1` on a string is not generally meaningful, and `std::string` has no `operator+(int)`). For `T` = a string type, these member functions are simply not usable (calling them is a compile error, "no matching member function"), rather than causing a runtime error. Note that this `requires` clause is independent of `checkType()`: it acts purely on the `AnyStandardString` concept, so it correctly also permits use for `bip::bigint` and any integral/floating type, and correctly excludes only actual standard strings.

5.5 `memvarPrinter()`: shared history-printing engine

```
void memvarPrinter (const memvarHistory& history,
                    std::ostream& os = std::cout,
                    const bool printReverse = false,
                    const std::string& separator = std::string(" ")) const {
  if ( history.empty() ) { os << "[]\n"; return; }
  auto printItem = [&separator, &os] (const T& item) -> void { os << item <<
      separator; };
  os << "[ ";
  if ( printReverse ) { std::for_each(history.crbegin(), history.crend(), printItem);
       }
  else { std::for_each(history.cbegin(), history.cend(), printItem); }
  os << "\b \b ]\n";
}
```

This `protected` helper is shared by `printHistoryData()` and `printReverseHistoryData()` (and is reused, on a different deque, for the timed variant's own printers). It writes `"[ "`, then every element of `history` followed by `separator`, in either forward (index 0 first, i.e. newest first) or reverse (oldest first) order, and finally writes the three-character sequence `"\b \b"` before the closing `"]"`.

Open Issue / Limitation

The trailing `"\b \b"` is a terminal-control trick: byte `0x08` (backspace) moves the cursor left one column, the following space overwrites whatever character is there, and the second backspace moves the cursor left again — the net visual effect, *on an interactive terminal that interprets control characters*, is to erase exactly the *last one character* that was written. This works correctly only when `separator` is exactly one character long (the default `" "`): after the last item the buffer holds `"...item "`; the trick erases that trailing single space, giving a clean `"[ a b c ]"`. If a caller supplies a multi-character separator (e.g. `", "`), only the *last character* of that separator is erased, leaving the rest behind: for items 1, 2, 3 with separator `", "` the raw stream would contain `"[ 1, 2, 3, "` before the trick, and the trick

only erases the trailing space (turning it into a lone space then moving back), so what reaches the terminal reads as `"[ 1, 2, 3, ]"` with the stray comma still present. In addition, if `os` is not an interactive terminal (e.g. a file, a string stream, or a piped/redirected process), the literal bytes `0x08 0x20 0x08` are written into the output as-is and are *not* interpreted as erasure by anything — they appear as raw (often invisible/unprintable) characters in the resulting file. This makes `printHistoryData`/`printReverseHistoryData` correct only for the specific case of single-character separators written to an interactive terminal; both other cases (non-default separator, or non-terminal stream) produce visibly incorrect or polluted output.

Design Note

The `history.empty()` branch (printing `"[]\n"`) is effectively unreachable through the class's own public API in ordinary single-threaded use: `memo_` always has at least one element after any constructor runs, and `clearHistory()` always re-inserts a default value in the same call that clears the deque, so no externally observable state has an empty history. It functions as defensive code against direct manipulation of `memo_` by a derived class, not as a reachable branch of normal operation.

5.6 Constructors and destructor

```
memvar() : memvarBase() {
  checkType();
  memo_.emplace_front(T{});
}
explicit memvar(const T& value,
                const capacityType historyCapacity = historyCapacityDefault_) :
memvarBase(historyCapacity) {
  checkType();
  if ( historyCapacity_ < minimumHistoryCapacity_ ) {
    throw std::invalid_argument("ERROR: The history capacity must be " +
      std::to_string(minimumHistoryCapacity_) + " at least, or more");
  }
  memo_.emplace_front(value);
}
virtual ~memvar() = default;
memvar(const memvar& rhs) = delete;
memvar(memvar&& rhs) = delete;
memvar& operator=(memvar&& rhs) = delete;
```

- The default constructor `memvar()` uses the default `memvarBase()` base (capacity fixed at 10), calls `checkType()`, and seeds the history with a single, value-initialized `T{}` (`0` for numeric types, an empty string for string types).
- The value constructor is `explicit` (preventing implicit conversion from a bare `T` to `memvar<T>`, e.g. in function-call argument passing), accepts an optional `historyCapacity` (defaulting to 10), forwards it to `memvarBase`, calls `checkType()`, and *then* explicitly re-checks `historyCapacity_ < minimumHistoryCapacity_`, throwing `std::invalid_argument` if the caller requested a capacity below 2. Only after this check does it seed the history with the supplied `value`.
- The destructor is `virtual` and defaulted, consistent with the class being designed as a base for `memvarTimed`.
- Copy construction and move construction/assignment are all deleted at the `memvar<T>` level (in addition to being deleted at the `memvarBase` level). Notably, copy *assignment*

(`operator=(const memvar&)`) is *not* deleted here — it is given a custom, non-defaulted definition, described next.

5.7 Conversion, call, and subscript operators

```
operator T() const { return getValue(); }
T operator()() const { return getValue(); }
T operator()(const capacityType index) const { return std::get<T>(getHistoryValue(
    index)); }
T operator[](const capacityType index) const { return std::get<T>(getHistoryValue(
    index)); }
```

- `operator T() const` is an implicit user-defined conversion from `memvar<T>` to a `T` *by value* (a copy of the current value); it is not marked `explicit`, so `memvar<T>` objects convert implicitly wherever a `T` is expected (function arguments, arithmetic expressions, `return` statements, etc.).
- `operator()()` (zero-argument call) is an explicit, equivalent way to fetch the current value; it is used pervasively by the free comparison/arithmetic operators (§5.12) instead of relying on the implicit conversion.
- `operator()(capacityType index)` and `operator[](capacityType index)` are *functionally identical* (same body: `std::get<T>(getHistoryValue(index))`) — two different syntaxes for indexed history access, both returning `T` *by value* (not a reference).

5.8 Assignment and compound-assignment operators

```
memvar& operator=(const T& rhs) { setValue(rhs); return *this; }
memvar& operator=(const memvar& rhs) { setValue(rhs.getValue()); return *this; }
memvar& operator+=(const T& rhs) { setValue(getValue() + rhs); return *this; }
memvar& operator+=(const memvar& rhs) { setValue(getValue() + rhs.getValue()); return
    *this; }
memvar& operator-=(const T& rhs) requires (!AnyStandardString<T>) { ... }
memvar& operator-=(const memvar& rhs) requires (!AnyStandardString<T>) { ... }
memvar& operator*=(const T& rhs) requires (!AnyStandardString<T>) { ... }
memvar& operator*=(const memvar& rhs) requires (!AnyStandardString<T>) { ... }
memvar& operator/=(const T& rhs) requires (!AnyStandardString<T>) { ... }
memvar& operator/=(const memvar& rhs) requires (!AnyStandardString<T>) { ... }
```

- `operator=(const T&)` and `operator=(const memvar&)` both push a *new* history entry via `setValue`; assignment is never a destructive overwrite of the current slot, it is always a history-preserving push. This is the central semantic difference between `memvar<T>` and a plain `T`: assignment grows history rather than mutating in place.
- `operator=(const memvar& rhs)` copies only *rhs's current value* (`rhs.getValue()`) into `*this`'s own history; it does not copy `rhs`'s entire history, nor its capacity. Combined with the deleted copy constructor, this means a `memvar<T>` can never be copy-*constructed*, but can be copy-*assigned* to (from another live `memvar<T>` of the same capacity or a different one) — and such assignment intentionally discards the source's history and capacity, transplanting only its present value.
- `operator+=` is defined unconditionally (no `requires` clause), so it is present for string `T` as well, relying on `std::string`'s own `operator+` (string concatenation) being well-formed. `operator-=`, `operator*=`, `operator/=` are all constrained with `requires (!AnyStandardString<T>)`,

since strings do not support subtraction, multiplication, or division; for a `memvar<std::string>`, only `operator=` and `operator+=` exist among the assignment-family operators.

- Move assignment `operator=(memvar&&)` is deleted (from §5.6), so there is no distinct "consume the moved-from object's value" assignment path; even with an rvalue `memvar<T>` on the right-hand side, overload resolution falls back to `operator=(const memvar&)` (a `const&` parameter binds to rvalues too), so the "move" actually performs a copy of the current value.

5.9 Increment and decrement operators

```
T operator++() requires (!AnyStandardString<T>) { return incr1(); }
T operator++([[maybe_unused]] int dummy) requires (!AnyStandardString<T>) {
  voidIncr1();
  return std::get<T>(getHistoryValue(1));
}
T operator--() requires (!AnyStandardString<T>) { return decr1(); }
T operator--([[maybe_unused]] int dummy) requires (!AnyStandardString<T>) {
  voidDecr1();
  return std::get<T>(getHistoryValue(1));
}
```

- Prefix `++mv`/`-mv` call `incr1()`/`decr1()`, which push the new value and *return the new value* (standard prefix semantics).
- Postfix `mv++`/`mv-` deliberately do *not* follow the conventional pattern of "save old value, mutate, return the saved old value". Instead they call `voidIncr1()`/`voidDecr1()` (which push the new value but return nothing), and then explicitly fetch `getHistoryValue(1)` — i.e., the entry that is now the *second*-newest, which, immediately after a push, is exactly the value that was current *before* the increment. The end effect (returning the pre-increment value) matches conventional postfix semantics, but it is achieved by re-reading history at index 1 rather than caching the old value in a local before mutating. A consequence: if `historyCapacity_` were ever as small as it can legally be (the enforced minimum of 2), `getHistoryValue(1)` remains valid (index 1 exists whenever size ≥ 2, which holds immediately after a second element is pushed into a deque with capacity ≥ 2).
- Like the helpers backing them, both prefix and postfix forms are constrained with `requires (!AnyStandardString<T>)` and are simply absent from the type's interface for string-valued `memvar`.

5.10 History introspection API

```
void printHistoryData(std::ostream& os = std::cout, const std::string& separator = "
    ") const;
void printReverseHistoryData(std::ostream& os = std::cout, const std::string&
    separator = " ") const;
capacityType getHistorySize() const noexcept { return static_cast<capacityType>(memo_
    .size()); }
virtual void clearHistory() { memo_.clear(); setValue(T{}); }
auto isHistoryFull() const noexcept { return static_cast<capacityType>(memo_.size())
    >= historyCapacity_; }
const auto& getMemVarHistory () const noexcept { return memo_; }
virtual historyValue getHistoryValue(const capacityType index) const {
  if ( (index < static_cast<capacityType>(memo_.size())) && (index >= 0) ) {
    return std::make_tuple(memo_.at(static_cast<size_t>(index)), false);
  }
  throw std::out_of_range(...);
```

```
}
auto getHistoryMinMax() const {
  const auto [min, max] = std::ranges::minmax_element(memo_);
  return std::make_tuple(*min, *max);
}
```

- `printHistoryData`/`printReverseHistoryData` are thin public wrappers over `memvarPrinter` (§5.5), printing newest-first and oldest-first respectively.
- `getHistorySize()` returns the current number of stored entries (which grows from 1 up to `historyCapacity_` and then stays there).
- `clearHistory()` is `virtual`; it clears `memo_` entirely and then calls `setValue(T{})`, which (because `setValue` is itself virtual) dispatches to the most-derived override — this is what lets `memvarTimed` transparently keep its time-tag history synchronized when the base's `clearHistory` is invoked (§6).
- `isHistoryFull()` compares current size against capacity; note it uses `>=` so it reports "full" exactly when the next `setValue` call would need to evict the oldest element.
- `getMemVarHistory()` returns a `const` reference to the underlying `std::deque<T>` directly, giving read-only access to the full container (including iteration, size, etc.) without going through `getHistoryValue`.
- `getHistoryValue(index)` is the `virtual`, bounds-checked, index-based accessor. It takes `index` as a *signed* `capacityType`, explicitly checks `index >= 0` (rejecting negative indices with the same `std::out_of_range` as an over-large index, rather than a different error, e.g. `std::invalid_argument`), and returns a `historyValue` = `std::tuple<T, bool>` where the `bool` is *always* `false` in every code path of the entire header — see the discussion below.
- `getHistoryMinMax()` runs `std::ranges::minmax_element` (a C++20 ranges algorithm) over `memo_` and returns a 2-tuple `(min, max)` *by value* (dereferencing the returned iterators). This requires `T` to support `operator<` (used internally by `minmax_element`); this requirement is not separately enforced by a `requires` clause or `static_assert` — if `T` lacks `operator<`, the error surfaces as a template instantiation failure inside `<algorithm>`/`<ranges>` at the call site of `getHistoryMinMax()`, not as a clear, class-level diagnostic.

Open Issue / Limitation

The error flag in `historyValue` is vestigial. `historyValue` is defined as `std::tuple<T, bool>`, and the same shape is repeated by `memvarTimed`'s `historyTimedValue` = `std::tuple<T, Time, bool>`. In every single construction of one of these tuples anywhere in the header (`getHistoryValue` in both `memvar` and `memvarTimed`, and `getHistoryTimedValue`), the boolean is hard-coded to `false`. There is no code path, anywhere in the header, that ever produces `true` for this flag. The type's shape strongly suggests an intended (but unimplemented) error/validity-signaling mechanism (for example, to flag a value obtained from a failed sensor read or a placeholder), but as shipped in this header it is dead weight: every consumer that pattern-matches or checks the second/third tuple element is checking a constant.

5.11 Free (non-member) helper

```
template <typename T>
T getHistoryValue(const memvar<T>& mv, const memvarBase::capacityType index) {
  return std::get<T>(mv.getHistoryValue(index));
}
```

A free function, still inside namespace `mv`, offering a slightly more convenient call syntax (`mv::getHistoryValue(m 3)`) that unwraps the tuple and returns just the `T` value, discarding the (always-`false`) flag. It shares its name with the *member* function `getHistoryValue`; the two are distinguished by argument-dependent lookup and signature (free function takes the `memvar<T>` as an explicit first argument), not by being overloads of one another in the usual sense — they live in different scopes (namespace scope vs. class scope).

5.12 Free relational and arithmetic operators

`memvar.h` defines, at *global* namespace scope (outside `mv`), an `operator«` for `memvar<T>`, and three overloads each of `operator==`, `operator>`, `operator<`, `operator>=`, `operator<=`, `operator+`, `operator-`, `operator*`, `operator/`: memvar-with-memvar, memvar-with-`T`, and `T`-with-memvar. All of them are expressed purely in terms of `operator()()` (the current value accessor), so none of them mutate history and none of them require friendship (`memvar<T>` exposes everything they need through its public interface).

```
1 template <typename T>
2 bool operator==(const mv::memvar<T>& mv1, const mv::memvar<T>& mv2) { return mv1() ==
      mv2(); }
3 template <typename T>
4 bool operator==(const mv::memvar<T>& mv, const T& v) { return mv() == v; }
5 template <typename T>
6 bool operator==(const T& v, const mv::memvar<T>& mv) { return v == mv(); }
```

Listing 5.1: Representative pattern (relational), repeated for each operator

- `operator!=` is *not* separately defined; because C++20 automatically rewrites/synthesizes `a != b` in terms of a visible `operator==(a, b)` when no explicit `operator!=` exists, `memvar<T>` objects (and `memvar<T>` vs. `T`) support `!=` "for free" from the C++20 language rule, not from any code in this header.
- `operator+` has *no* `requires` clause and thus is available for `AnyStandardString` types too (relying on `std::string`'s `operator+` for concatenation); `operator-`, `operator*`, `operator/` are all constrained with `requires (!mv::AnyStandardString<T>)`, mirroring the constraints on the corresponding compound-assignment operators.
- All four arithmetic free operators *return a plain `T`*, not a `memvar<T>`. Consequently, expressions like `mv1 + mv2` or `mv + 5` *do not* record anything into either operand's history; only the compound-assignment forms (`+=` etc.) or plain assignment do. This is a deliberate, meaningful design choice: pure arithmetic expressions are value-level and side-effect-free with respect to history, matching ordinary arithmetic operator expectations, while only explicit assignment-family operators are stateful.
- Relational and arithmetic operators are defined only between a `memvar<T>` and *the same* `T` (or another `memvar<T>` of the same `T`); there is no support for comparing/combining a `memvar<T>` with a `memvar<U>` for `T != U`, nor for comparing a `memvar<T>` against a value of a type merely convertible to `T` without an exact template argument match (template argument deduction requires an exact, or at least non-ambiguous deducible, match for `T` in the free-function templates).

Chapter 6

memvarTimed<T, Time, Clock>: Time-Tagged History

6.1 Purpose

`memvarTimed` is declared `final` and publicly derives from `memvar<T>`, adding a second, parallel `std::deque` that records *when* each value in the value history (`memo_`, inherited from `memvar<T>`) was set, expressed as a duration since a per-object "epoch" time point captured at construction (or at the most recent `clearHistory()`).

```
template <typename T,
          typename Time = std::chrono::nanoseconds,
          typename Clock = std::chrono::steady_clock>
class memvarTimed final : public memvar<T> {
public:
  using historyTimedValue = std::tuple<T, Time, bool>;
  ...
private:
  using memvarTimeHistory = std::deque<std::chrono::time_point<Clock, Time>>;
  memvarTimeHistory timeMemo_ {};
  std::chrono::time_point<Clock, Time> memvarEpoch_ {};
  void setTimeTag() { timeMemo_.emplace_front(Clock::now()); }
  void setValue(const T& value) override { ... }
};
```

- Two template parameters have defaults: `Time` defaults to `std::chrono::nanoseconds` (the duration unit that time tags are reported in), and `Clock` defaults to `std::chrono::steady_clock` (a monotonic clock, appropriate for measuring elapsed durations, as opposed to `system_clock`, which can jump with wall-clock adjustments).
- `timeMemo_` is a separate `std::deque<std::chrono::time_point<Clock, Time»`, kept, by construction of the code, in lock-step with `memo_`: every push to one is matched by exactly one push to the other, in the same relative order (front = newest), so that index i in `timeMemo_` corresponds to index i in `memo_`.
- `memvarEpoch_` records the clock reading at the moment the object was constructed (or last `clearHistory()`-ed); all reported time tags are *relative* to this epoch (via `getTimeTag`), not absolute clock readings.
- `setValue` is `private` in `memvarTimed` (though it `override`s a `protected` virtual in `memvar<T>`). C++ access control for virtual functions is checked against the *static* type used at the call

site, not the function's access level in the most-derived class; so calling `setValue` through a `memvar<T>&` or from within `memvar<T>`'s own member functions (which is exactly how it is always invoked — see `clearHistory`, `operator=`, etc., all defined in `memvar<T>`) is permitted (checked against `memvar<T>`'s `protected` access), even though `memvarTimed` itself could not call `this->memvarTimed::setValue(...)` qualified through its own narrower access from outside the class.

6.2 Constructors

```
memvarTimed() : memvar<T>() {
  memvarEpoch_ = Clock::now();
  timeMemo_.emplace_front(memvarEpoch_);
}
explicit memvarTimed(const T& value,
                     const memvarBase::capacityType historyCapacity = memvarBase::
                         historyCapacityDefault_) :
memvar<T>(value, historyCapacity) {
  memvarEpoch_ = Clock::now();
  timeMemo_.emplace_front(memvarEpoch_);
}
memvarTimed(const memvarTimed& rhs) = delete;
memvarTimed(memvarTimed&& rhs) = delete;
memvarTimed& operator=(memvarTimed&& rhs) = delete;
```

Both constructors first fully construct the `memvar<T>` base subobject (which, in turn, runs `checkType()` and seeds `memo_` with one entry, and — for the value constructor — validates the requested capacity), *then* capture `memvarEpoch_ = Clock::now()` and push that same instant as the first entry of `timeMemo_`. Because the base's `setValue` is not used for this first entry (the base constructor directly `emplace_front`s into `memo_`), the derived class must, and does, separately seed `timeMemo_` with a matching first entry so the two deques start in sync at size 1. Copy construction, move construction, and move assignment are all deleted, mirroring `memvar<T>`; copy *assignment* is again separately (and differently) defined below.

6.3 Assignment operators and value accessors

```
operator T() const { return memvar<T>::getValue(); }
T operator()() const { return memvar<T>::getValue(); }
T operator()(const memvarBase::capacityType index) const { return std::get<T>(
    getHistoryValue(index)); }
memvarTimed& operator=(const T& rhs) { setValue(rhs); return *this; }
memvarTimed& operator=(const memvarTimed& rhs) { setValue(rhs.getValue()); return *
    this; }
memvarTimed& operator+=(const T& rhs) { setValue(memvar<T>::getValue() + rhs); return
     *this; }
... (analogous -=, *=, /= with requires (!AnyStandardString<T>))
```

`memvarTimed` redeclares its own `operator T()`, `operator()()`, `operator()(index)`, `operator=`, and the compound-assignment family, all with bodies essentially identical in effect to `memvar<T>`'s own (they call the same, non-virtual `memvar<T>::getValue()`/inherited `setValue`, just spelled out again). Because none of `memvar<T>`'s `operator()`, `operator T()`, or `operator=` are `virtual`, these derived declarations *hide* (not override) the base versions when called through a `memvarTimed` object or reference/pointer typed as `memvarTimed`; when called through a `memvar<T>&` reference bound to a `memvarTimed` object (e.g. inside the free operators of §5.12, which take `const mv::memvar<T>&` parameters), the *base* versions are invoked instead (no dynamic dispatch), though

they compute the same result since both ultimately call `getValue()` on the same shared `memo_`. `operator[]` is *not* redeclared in `memvarTimed` (unlike `operator()(index)`), so subscripting a `memvarTimed` object with `[]` uses the inherited, non-virtual `memvar<T>::operator[]`, which in turn calls `this->getHistoryValue(index)` — and because `getHistoryValue` *is* virtual and *is* overridden in `memvarTimed`, `operator[]` on a `memvarTimed` object does correctly dispatch to `memvarTimed::getHistoryValue`, even though `operator[]` itself is not overridden.

6.4 Time-tag accessors and printers

```
Time getTimeTag(const size_t index = 0) const {
  return std::chrono::duration_cast<Time>(timeMemo_.at(index) - memvarEpoch_);
}
void printHistoryTimedData(...) const;
void printReverseHistoryTimedData(...) const;
```

- `getTimeTag(index)` returns, as a `Time` duration, how long after `memvarEpoch_` the value at history slot `index` was recorded. It defaults to `index = 0` (the most recent value's time tag).
- Its parameter type is `size_t` (unsigned), unlike essentially every other indexing parameter in the library, which uses the signed `memvarBase::capacityType` (`int64_t`) — for example, `operator()(capacityType)`, `operator[](capacityType)`, and `getHistoryValue(capacityType)` all take signed indices and explicitly reject negative ones with a clear `std::out_of_range`. A negative value passed to `getTimeTag`'s `size_t` parameter instead silently wraps to a very large unsigned value, which will still, in the end, cause `timeMemo_.at(index)` to throw `std::out_of_range` — so the failure mode is still an exception, but the type signature itself admits an inconsistency in the public API's indexing convention.
- `printHistoryTimedData`/`printReverseHistoryTimedData` do *not* reuse `memvarPrinter`; they are hand-written loops that directly index both `memvar<T>::memo_` (accessible because it is `protected` in the base and `memvarTimed` derives from `memvar<T>`) and `timeMemo_` (via `getTimeTag`) in lock-step, printing `"[timeTag:value]"` pairs separated by a (default) newline separator, forward or reverse respectively. They loop over indices `0` to `memo_.size() - 1`, so they assume (without re-checking) that `timeMemo_` has at least as many elements as `memo_` at every index accessed.

6.5 `clearHistory()` override

```
void clearHistory() override {
  memvar<T>::clearHistory();
  timeMemo_.clear();
  memvarEpoch_ = Clock::now();
  timeMemo_.emplace_front(memvarEpoch_);
}
```

This override first calls the *base* class's `clearHistory()` *directly* (not virtually, via qualified name `memvar<T>::clearHistory()`), which itself does `memo_.clear(); setValue(T{});` — and because that internal `setValue` call is a virtual call made from within a `memvar<T>` member function on a `memvarTimed` object, it dynamically dispatches to `memvarTimed::setValue`, which (a) calls `setTimeTag()`, pushing one entry into `timeMemo_` and possibly trimming it against the (still-current, not yet reset) `historyCapacity_`, and then (b) calls `memvar<T>::setValue(value)` to push the fresh default value into `memo_`. *After* that whole chain completes and control returns to `memvarTimed::clearHistory()`, the code then `timeMemo_.clear()`s (discarding the entry that

step (a) just added), resets `memvarEpoch_` to a fresh `Clock::now()`, and re-seeds `timeMemo_` with exactly that new epoch as its sole entry.

Open Issue / Limitation

Redundant work and a subtle timing window in `memvarTimed::clearHistory()`. The sequence above means a call to `clearHistory()` on a `memvarTimed` object performs *two* clock reads (`Clock::now()` is called once inside the virtually-dispatched `setValue` during the base call, and again explicitly afterward) and pushes into `timeMemo_` *twice* (once via `setTimeTag()` during the base call, which is then immediately thrown away by the following `timeMemo_.clear()`). Functionally the end state is still correct (one entry in each deque, in sync, both representing the fresh default value), but the implementation does strictly more work than necessary, and the discarded first time-tag means the object's `memvarEpoch_` is set *after* `memo_` was already updated with the new value, rather than atomically with it — an external observer with a reference to the object could not realistically catch this since the whole operation is not re-entrant/thread-safe to begin with (see the general thread-safety discussion in the Limitations chapter), but it is worth noting as an implementation inefficiency and an example of the base/derived `setValue` virtual-dispatch chain producing work that the derived `clearHistory` then undoes.

6.6 `getHistoryValue` override and `getHistoryTimedValue`

```
1 memvar<T>::historyValue getHistoryValue(const memvarBase::capacityType index) const
      override {
2   if ( (index < static_cast<memvarBase::capacityType>(memvar<T>::memo_.size())) && (
        index >= 0) ) {
3     return std::make_tuple(memvar<T>::memo_.at(static_cast<size_t>(index)), false);
4   }
5   throw std::out_of_range(...);
6 }
7 historyTimedValue getHistoryTimedValue(const memvarBase::capacityType index) const {
8   return std::make_tuple(std::get<0>(getHistoryValue(index)), getTimeTag(static_cast<
        size_t>(index)), false);
9 }
```

`memvarTimed` overrides `getHistoryValue` with logic that is, in fact, *value-identical* to the inherited `memvar<T>::getHistoryValue` (same bounds check, same tuple shape, same hard-coded `false`); this override exists to give `memvarTimed` an implementation textually independent of the base's private/protected internals it already has direct access to, rather than to change behavior. `getHistoryTimedValue` is a *non-virtual*, `memvarTimed`-only addition that returns the richer 3-tuple `(value, timeTag, false)`, built by calling the (virtual, but here statically resolved since called as a plain member function on `*this`) `getHistoryValue` for the value and `getTimeTag` for the elapsed duration.

6.7 Free `getHistoryValue` and `operator«` for `memvarTimed`

```
1 template <typename T>
2 T getHistoryValue(const memvarTimed<T>& mvt, const memvarBase::capacityType index) {
3   return std::get<T>(mvt.getHistoryValue(index));
4 }
```

```
1 template <typename T>
2 std::ostream& operator<<(std::ostream& os, const mv::memvarTimed<T>& mvt) {
3   return os << "[" << mvt.getTimeTag().count() << ":" << mvt() << "]";
```

```
4 }
```

The free `mv::getHistoryValue` overload for `memvarTimed<T>` only takes a `T` template parameter, *not* `Time`/`Clock`, so it only matches `memvarTimed<T, Time, Clock>` when `Time` and `Clock` are left at their *defaults* (`std::chrono::nanoseconds`, `std::chrono::steady_clock`); a `memvarTimed<int, std::chrono::milliseconds>` would not match this template's single deduced parameter `T` against the two-or-three-parameter class template `memvarTimed<T, Time, Clock>` the way it is written here.

Open Issue / Limitation

`template <typename T> T getHistoryValue(const memvarTimed<T>& mvt, ...)` declares its parameter as `memvarTimed<T>`, which is really `memvarTimed<T, std::chrono::nanoseconds, std::chrono::steady_clock>` (defaults filled in) once written out in full — template argument deduction from a function call can only deduce `T` here, and the class template's other two parameters are fixed at their defaults by this signature, *not* independently deduced. As a result, this convenience free function silently fails to match (falls out of overload resolution, not a compile error by itself, but simply "no viable overload" if it were the only candidate) for any `memvarTimed` instantiated with a non-default `Time` or `Clock`, while the member function `getHistoryValue` on the object itself always works regardless of `Time`/`Clock`. The same restriction applies to the global `operator«` for `memvarTimed<T>` shown above: it only matches default-`Time`/`Clock` instantiations, so streaming a `memvarTimed<int, std::chrono::milliseconds>` with `operator«` would fail to find this overload (and, absent another matching overload, fail to compile) even though the class itself fully supports non-default `Time`/`Clock` parameters everywhere else.

Chapter 7

Usage Examples

This chapter presents complete, self-contained usage examples reflecting exactly the public interface described above. Each example assumes `#include "memvar.h"` (which transitively includes `is_string.h`) and, where relevant, `using namespace mv;` or explicit `mv::` qualification as shown.

7.1 Basic construction, assignment, and value access

```cpp
#include "memvar.h"
#include <iostream>

int main() {
  mv::memvar<int> counter; // default: value 0, capacity 10
  std::cout << counter() << "\n"; // 0 (operator())
  std::cout << counter << "\n"; // 0 (implicit operator T())

  counter = 5; // pushes 5 (history: [5, 0])
  counter = 12; // pushes 12 (history: [12, 5, 0])
  counter += 3; // pushes 15 (history: [15, 12, 5, 0])

  std::cout << "current: " << counter() << "\n"; // current: 15
  std::cout << "previous: " << counter[1] << "\n"; // previous: 12
  std::cout << "oldest: " << counter[counter.getHistorySize() - 1] << "\n"; // oldest
      : 0
}
```

Listing 7.1: Default construction, assignment, and read-back

7.2 Explicit construction with a custom history capacity

```cpp
mv::memvar<double> temperature(21.5, 3); // capacity 3, seeded with 21.5

temperature = 22.0; // history: [22.0, 21.5]
temperature = 23.5; // history: [23.5, 22.0, 21.5] (size == capacity)
temperature = 19.0; // history: [19.0, 23.5, 22.0] (21.5 evicted)

std::cout << temperature.getHistorySize() << "\n"; // 3
std::cout << temperature.isHistoryFull() << "\n"; // 1 (true)
```

Listing 7.2: Custom capacity and capacity-driven eviction

7.3 Invalid capacity: exception on construction

```
try {
  mv::memvar<int> broken(10, 1); // capacity 1 < minimumHistoryCapacity_ (2)
} catch (const std::invalid_argument& e) {
  std::cerr << "Rejected: " << e.what() << "\n";
  // Rejected: ERROR: The history capacity must be 2 at least, or more
}
```

Listing 7.3: Requesting a capacity below the enforced minimum of 2

7.4 String-valued memvar

```
mv::memvar<std::string> name("Ada");
name = "Ada Lovelace";
name += ", Countess of Lovelace"; // string concatenation via operator+=

std::cout << name() << "\n";
// name++; // COMPILE ERROR: operator++ excluded by
                        // requires (!AnyStandardString<T>)
// name -= "x"; // COMPILE ERROR: operator-= excluded likewise
// mv::memvar<int> n = name; // COMPILE ERROR: no viable conversion int<-string
```

Listing 7.4: `memvar<std::string>`: allowed operators only

7.5 Wide and Unicode string memvars and printing

```
mv::memvar<std::wstring> label(L"caf\u00e9");
label = L"na\u00efve";
std::wcout; // (not used: memvar<T>'s operator<< is narrow-stream)
std::cout << label << "\n"; // uses the header's std::ostream& operator<<(std::
    wstring)
                                // to transcode to UTF-8 before printing
```

Listing 7.5: Streaming a `memvar<std::wstring>`

7.6 Prefix/postfix increment and decrement

```
mv::memvar<int> tally(0, 5);

int a = ++tally; // tally becomes 1; a == 1; history: [1, 0]
int b = tally++; // tally becomes 2; b == 1 (pre-increment value); history: [2, 1, 0]
int c = --tally; // tally becomes 1; c == 1; history: [1, 2, 1, 0]
int d = tally--; // tally becomes 0; d == 1 (pre-decrement value); history: [0, 1, 2,
     1, 0]

std::cout << a << " " << b << " " << c << " " << d << "\n"; // 1 1 1 1
```

Listing 7.6: Increment/decrement semantics and history growth

7.7 History iteration, printing, min/max

```cpp
mv::memvar<int> series(1, 6);
series = 4; series = 9; series = 2; series = 7;
// history (newest -> oldest): [7, 2, 9, 4, 1]

series.printHistoryData(); // [ 7 2 9 4 1 ]
series.printReverseHistoryData(); // [ 1 4 9 2 7 ]

auto [minVal, maxVal] = series.getHistoryMinMax();
std::cout << minVal << " " << maxVal << "\n"; // 1 9

for (int v : series.getMemVarHistory()) {
  std::cout << v << ' '; // 7 2 9 4 1 (deque iteration order == storage order)
}
```

Listing 7.7: Printing and reducing the stored history

7.8 Indexed access and out-of-range handling

```cpp
mv::memvar<int> v(100, 4);
v = 200; v = 300; // history: [300, 200, 100]

std::cout << v[0] << "\n"; // 300 (current)
std::cout << v(1) << "\n"; // 200
try {
  std::cout << v[10] << "\n"; // out of range: throws std::out_of_range
} catch (const std::out_of_range& e) {
  std::cerr << e.what() << "\n";
}
try {
  std::cout << v(-1) << "\n"; // negative index: also throws std::out_of_range
} catch (const std::out_of_range& e) {
  std::cerr << e.what() << "\n";
}
```

Listing 7.8: `operator[]`, `operator()(index)`, and bounds errors

7.9 Free function `getHistoryValue` and `historyValue` tuple

```cpp
mv::memvar<int> v(5, 4);
v = 6;
mv::memvar<int>::historyValue hv = v.getHistoryValue(0);
auto [val, errFlag] = hv;
std::cout << val << " " << std::boolalpha << errFlag << "\n"; // 6 false (errFlag
    always false)

int justValue = mv::getHistoryValue(v, 1); // free-function convenience wrapper
std::cout << justValue << "\n"; // 5
```

Listing 7.9: Reading the tuple form directly

7.10 Clearing history

```cpp
mv::memvar<int> v(42, 4);
v = 43; v = 44; // history: [44, 43, 42] (size 3)
v.clearHistory();
```

```
std::cout << v.getHistorySize() << " " << v() << "\n"; // 1 0
```

Listing 7.10: `clearHistory()` resets to a single default-valued entry

7.11 Copy semantics: what is and is not allowed

```
mv::memvar<int> a(1, 5);
// mv::memvar<int> b = a; // COMPILE ERROR: copy constructor is deleted
// mv::memvar<int> c(std::move(a)); // COMPILE ERROR: move constructor is deleted

mv::memvar<int> d(0, 5);
d = a; // OK: copy ASSIGNMENT is user-defined (pushes a's current value into d)
// d's own capacity (5) and prior history are unaffected except for the new push;
// a's history is untouched.
```

Listing 7.11: Deleted copy/move construction vs. defined copy assignment

7.12 Comparing and combining `memvar` values

```
mv::memvar<int> x(10, 5), y(20, 5);

bool lt = x < y; // true (10 < 20), via operator<(memvar,memvar)
bool eq = (x == 10); // true, via operator==(memvar, T)
bool eq2 = (10 == x); // true, via operator==(T, memvar)

int sum = x + y; // 30 (plain int, NOT a memvar<int>; no history change to x or y)
int diff = y - x; // 10
int prod = x * 2; // 20
int quot = y / x; // 2

std::cout << x.getHistorySize() << "\n"; // still 1: x + y did not push anything into
     x or y
```

Listing 7.12: Relational and arithmetic free operators

7.13 Using `memvarTimed`

```
#include "memvar.h"
#include <thread>
#include <chrono>

mv::memvarTimed<int> reading(0, 5); // Time = nanoseconds, Clock = steady_clock (
    defaults)

std::this_thread::sleep_for(std::chrono::milliseconds(10));
reading = 7;
std::this_thread::sleep_for(std::chrono::milliseconds(10));
reading = 13;

std::cout << reading() << "\n"; // 13
std::cout << reading.getTimeTag().count() << "\n"; // nanoseconds since epoch for the
     latest value
std::cout << reading.getTimeTag(1).count() << "\n"; // time tag for the previous
    value (7)

```

```
reading.printHistoryTimedData();
// { --- begin ---
// [TimeTag:Value]
// [<ns3>:13]
// [<ns2>:7]
// [<ns1>:0]
// --- end --- }

std::cout << reading << "\n"; // uses memvarTimed's operator<<: "[<ns>:13]"
```

Listing 7.13: Time-tagged history: construction, updates, and inspection

```
using MsClockVar = mv::memvarTimed<double, std::chrono::milliseconds, std::chrono::
    system_clock>;
MsClockVar price(99.99, 4);
price = 101.5;
std::cout << price.getTimeTag().count() << " ms\n";

// The free convenience functions below will NOT match MsClockVar, because they only
// deduce a single template parameter T against memvarTimed<T> (defaults filled in):
//
// mv::getHistoryValue(price, 0); // no matching overload
// std::cout << price; // no matching operator<< overload (unless one
// // more specific is written for this instantiation)
//
// Member access still works fully, regardless of Time/Clock:
std::cout << std::get<0>(price.getHistoryTimedValue(0)) << "\n"; // 101.5
```

Listing 7.14: A non-default `Time`/`Clock` instantiation

7.14 Using `mv::type<T>()` and `mv::is_string_v<T>` directly

```
#include "is_string.h"
#include <iostream>

std::cout << mv::type<std::string>() << "\n"; // e.g. "std::string" (demangled)
std::cout << mv::type<mv::memvar<int>>() << "\n"; // demangled memvar<int>
    instantiation name

static_assert(mv::is_string_v<std::string>);
static_assert(mv::is_string_v<const std::wstring&>); // remove_cvref_t applied
static_assert(!mv::is_string_v<std::string_view>); // string_view NOT recognized
static_assert(!mv::is_string_v<const char*>);
```

Listing 7.15: Diagnostics and trait usage independent of `memvar`

7.15 A custom "bigint-like" extension type

```
// user_bigint.h (hypothetical, illustrating the extension point)
namespace myapp { class Decimal128 { /* ... arithmetic ops, operator<<, operator< ...
     */ }; }

namespace mv {
template <>
struct is_bigint<myapp::Decimal128> : std::true_type {};
}

```

```
// now allowed:
mv::memvar<myapp::Decimal128> price{myapp::Decimal128{"19.99"}, 5};
```

Listing 7.16: Opting a user type into `memvar` via `is_bigint`

Chapter 8

Edge Cases

8.1 Type rejected only at first construction, not at first naming

```
using Bad = mv::memvar<std::vector<int>>; // compiles: no object constructed yet
// Bad x; // COMPILE ERROR here: checkType()'s
                                          // static_assert fires only now
```

Because `checkType()`'s `static_assert` lives inside a member function body that is only instantiated on odr-use, merely creating a type alias, taking `sizeof(Bad)`, or declaring (but never constructing) a variable of pointer/reference type to `Bad` does not trigger the check.

8.2 Multi-character separators in printers corrupt output

As detailed in §5.5, calling `printHistoryData(std::cout, ", ")` produces a trailing, un-erased comma before the closing bracket, because the erasure trick only removes exactly one trailing character:

```
mv::memvar<int> v(1, 4);
v = 2; v = 3;
v.printHistoryData(std::cout, ", ");
// actual terminal output: [ 3, 2, 1, ] <-- stray trailing comma
// (a single-character separator, e.g. " " or ",", does not exhibit this problem)
```

8.3 Printing history to a non-terminal stream

```
std::ostringstream oss;
mv::memvar<int> v(1, 4);
v = 2;
v.printHistoryData(oss);
std::string s = oss.str();
// s contains the literal bytes 0x08 0x20 0x08 immediately before "]\n" --
// no erasure occurs since ostringstream does not interpret control characters.
```

8.4 Division and the absence of divide-by-zero protection

```
mv::memvar<int> v(10, 4);
v /= 0; // integer division by zero: undefined behavior (typically SIGFPE
                    // on common platforms), NOT caught or translated into an
                        exception
```

```
                    // by memvar<T> -- there is no guard in operator/= or checkType
                        ().

mv::memvar<double> d(10.0, 4);
d /= 0.0; // well-defined for floating point: d() becomes +inf, no crash,
                    // no exception -- IEEE 754 semantics apply as-is.
```

8.5 Postfix increment near the enforced minimum capacity

```
mv::memvar<int> v(0, 2); // minimum legal capacity
int old = v++; // history before: [0]; after voidIncr1(): [1, 0] (size 2, at cap)
                            // getHistoryValue(1) == (0, false) -> old == 0. Works
                                correctly
                            // exactly because capacity >= 2 is enforced by the
                                constructor.
```

Had the minimum capacity not been enforced (e.g. if a future maintainer bypassed `memvar`'s constructor and constructed a `memvarBase` subobject with capacity 1 directly, see the issue noted in §9.1), a capacity-1 history would only ever contain 1 element after `voidIncr1()`'s internal eviction, and `getHistoryValue(1)` inside postfix `operator++` would throw `std::out_of_range` instead of returning the pre-increment value.

8.6 `memvarTimed` with a non-default `Time`/`Clock` and the free helpers

As shown in the `MsClockVar` example above, the free `mv::getHistoryValue(memvarTimed<T> &, index)` function and the free `operator«` for `memvarTimed<T>` are only usable for instantiations that keep `Time` and `Clock` at their class-template defaults; both silently fall out of overload resolution (compile error: no matching function) for any other `Time`/`Clock` combination, even though every member function of `memvarTimed` itself works uniformly regardless of these parameters.

8.7 Slicing-free but dispatch-inconsistent access through base references

```
mv::memvarTimed<int> t(1, 5);
mv::memvar<int>& base = t; // reference binding, NOT slicing (no object copy)
base = 2; // calls memvar<int>::operator=(const T&), which calls
                                    // setValue(2) -- virtual dispatch DOES reach
                                    // memvarTimed::setValue, so the time tag IS updated
std::cout << base() << "\n"; // 2 -- memvar<int>::operator()() is non-virtual, but
                                    // both classes' operator() read the same memo_, so
                                        the
                                    // *value* is correct even though it is not a "timed
                                        " call
std::cout << base[0] << "\n"; // uses memvar<int>::operator[], which calls the *
    virtual*
                                    // getHistoryValue() -- this DOES dispatch to
                                    // memvarTimed::getHistoryValue
```

This demonstrates that even though several accessor functions are non-virtual and thus do not dynamically dispatch when called through a base reference, the object's *observable value* remains correct because both classes ultimately read the same shared `memo_`; only `memvarTimed`-specific data (the time tags) is inaccessible through a `memvar<int>&` reference, which is expected, since `memvar<T>` exposes no time-tag API at all.

Chapter 9

Limitations, Constraints, and Open Issues

This chapter consolidates issues raised throughout the guide and adds several that only become visible when considering the header as a whole.

9.1 Type-safety and validation

- **Deferred type checking.** `checkType()`'s `static_assert` fires only when a `memvar<T>` constructor is instantiated (first construction), not when the class template is merely named with a disallowed `T`. A `requires` clause on the class template declaration itself would enforce the constraint earlier and more idiomatically for C++20.
- **Capacity validation lives in one constructor only.** The check `historyCapacity_ < minimumHistoryCapacity_` is performed exclusively in `memvar`'s value constructor. `memvarBase`'s one-argument constructor, which actually stores the capacity, performs no validation and is `noexcept`; any future derived class that constructs a `memvarBase` subobject directly (bypassing `memvar<T>`'s constructor logic) would not inherit this protection automatically.
- **Duplicated string-type lists.** The set of five recognized standard string types is written out independently in `mv::is_string` (`is_string.h`) and in `mv::AnyStandardString` (`memvar.h`); the two are not defined in terms of one another, so they can drift out of sync if one is edited without the other.
- **No divide-by-zero guard.** Integral `operator/=`/`operator/` perform no zero-check; integer division by zero is undefined behavior on the underlying platform, uncaught by any part of the library.
- `getHistoryMinMax` **requires** `operator<` **implicitly.** There is no `static_assert`/`requires` documenting or enforcing that `T` support `operator<`; failure surfaces as a deep template-instantiation error from `<algorithm>`/`<ranges>` rather than a clear, local diagnostic.

9.2 The unused error flag

Every `historyValue` (`std::tuple<T,bool>`) and `historyTimedValue` (`std::tuple<T,Time,bool>`) produced anywhere in the header hard-codes its trailing `bool` to `false`. No code path ever sets it to `true`. As shipped, this is an unused/vestigial field that nonetheless is part of the observable public API (`historyValue` is a public type alias, and every consumer decomposing the tuple must account for the flag).

9.3 Output/formatting correctness

- The backspace-based trailing-separator trimming in `memvarPrinter` (used by `printHistoryData`/`printRev` is correct only for single-character separators written to an interactive terminal; it corrupts output for multi-character separators and leaves raw control bytes in non-terminal streams (files, string streams, pipes).
- The `std::wstring` stream-insertion overload assumes `wchar_t` represents UTF-32 code points (true on Linux/most Unix platforms, false on Windows, where `wchar_t` is UTF-16), making that specific overload's correctness platform-dependent.
- The `std::u8string` overload relies on `c_str()`'s NUL terminator rather than the string's stored `size()`, so any `std::u8string` containing an embedded NUL code unit before its logical end would be truncated when printed.
- All Unicode-conversion overloads depend on `std::wstring_convert` and the `std::codecvt_utf8*` facets, deprecated since C++17 with no standard replacement as of C++20; this is a forward-compatibility risk should a future standard library remove them.

9.4 Concurrency

Open Issue / Limitation

Nothing in `memvarBase`, `memvar<T>`, or `memvarTimed<T,Time,Clock>` performs any synchronization (no mutexes, atomics, or other coordination). `setValue` performs a multi-step, non-atomic sequence (push front, conditionally pop back); `memvarTimed::setValue` performs an even longer multi-step sequence across two separate deques (`timeMemo_` and, via the base call, `memo_`). Any concurrent access to the same `memvar`/`memvarTimed` object from multiple threads without external synchronization (even a mix of one writer and one reader) is a data race and undefined behavior by the C++ memory model; the library provides no internal protection against this.

9.5 Exception safety

Open Issue / Limitation

`memvarTimed::setValue` first calls `setTimeTag()` (pushing into `timeMemo_` and possibly popping from it), and *then* calls `memvar<T>::setValue(value)` (pushing into `memo_` and possibly popping from it). If constructing/copying the new `T` into `memo_.emplace_front(value)` throws (plausible for a `T` whose copy/move constructor can throw, e.g. a string requiring allocation, or a user-supplied `bigint`-like type), the `timeMemo_` deque has already been updated (and possibly trimmed) while `memo_` was not, leaving the two deques with *different sizes* — breaking the invariant, relied upon throughout `memvarTimed` (e.g. in `printHistoryTimedData`, which loops up to `memo_.size()` and indexes `timeMemo_` at the same indices), that the two histories stay in lock-step. The library offers no strong exception guarantee here, and no code detects or repairs a desynchronization after such a partial failure.

9.6 API consistency

- **Signed vs. unsigned indices.** Most indexing APIs (`operator()(index)`, `operator[]`, `getHistoryValue`) use the signed `memvarBase::capacityType` (`int64_t`) and explicitly reject negative values with `std::out_of_range`. `memvarTimed::getTimeTag(size_t index`

`= 0)` instead takes an unsigned `size_t`, breaking this convention; a negative argument silently wraps rather than being rejected by an explicit, matching check (it still eventually throws `std::out_of_range` via `deque::at`, but via a different code path and without the deliberate "≥ 0" guard used elsewhere).

- **Free helper functions with narrower applicability than the member functions they wrap.** The free `mv::getHistoryValue(const memvarTimed<T>&, capacityType)` and the global `operator«(std::ostream&, const mv::memvarTimed<T>&)` both only deduce `T`, implicitly fixing `Time` and `Clock` at their class-template defaults; neither works for a `memvarTimed` instantiated with an explicit `Time` and/or `Clock`, even though all member functions of the class support arbitrary `Time`/`Clock` uniformly.
- **Duplicated, near-identical accessor bodies.** `memvarTimed` redeclares `operator T()`, `operator()()`, `operator=`, and the compound-assignment family with bodies that compute the same result as the (non-virtual) base versions, rather than reusing them; this is required only because the base versions are not virtual, so relying on inheritance alone would not let `memvarTimed`'s own overloads be preferred when called on a `memvarTimed`-typed expression (needed for overload resolution/hiding to work as intended), but it does mean the same logic exists in two places.
- **No comparison/arithmetic operators between differing `T`.** All free relational and arithmetic operators require both operands to resolve to the exact same `T` (or a `memvar<T>` of that `T`); comparing/combining `memvar<int>` with `memvar<long>`, or with a plain `long`, is not supported directly and requires an explicit cast by the caller.
- **No relational/arithmetic operators for `memvarTimed` beyond what it inherits.** `memvarTimed` does not declare its own relational operators; it relies on implicit conversion (`operator T()`) or the inherited comparison templates (which take `const mv::memvar<T>&`, matching via the base class) for comparisons.

9.7 Extensibility and completeness

- **Windows/MSVC name demangling is unsupported.** `mv::type<T>()`'s demangling branch is guarded only by `#ifndef NO_DEMANGLE` and unconditionally `#includes <cxxabi.h>` in that branch; on a toolchain without that header (e.g. MSVC without a compatibility shim), the translation unit fails to compile unless `NO_DEMANGLE` is defined by the build, in which case names are left mangled.
- **`NO_DEMANGLE`-guarded function bodies are an ODR hazard** across translation units compiled with inconsistent definitions of that macro, as discussed in Chapter 2.
- **The `bip::bigint` extension point depends on an external, unseen header.** `bip::bigint` is only forward-declared; `is_bigint<bip::bigint>` is specialized to `std::true_type` despite `bip::bigint` never being completed in these files. Constructing an actual `mv::memvar<bip::bigint>` additionally requires `bip::bigint` to be a complete type by the time `memvar<bip::bigint>`'s constructor is instantiated (since `memvarHistory = std::deque<bip::bigint>` must store, copy, and destroy real `bip::bigint` objects), which in turn requires including the (unseen) `bigint.h` and depends on `bigint.h` providing whatever arithmetic/streaming/comparison operators the specific `memvar<bip::bigint>` usage needs (e.g. `operator+`, `operator<`, `operator«`) — none of which is guaranteed or checked by `memvar.h` itself beyond the coarse `is_bigint_v` admissibility flag.
- **No serialization/comparison hooks beyond `operator<`.** Features like `getHistoryMinMax` assume `operator<`; there is no analogous `static_assert` surfacing this requirement to a

`memvar<T>` user ahead of time for a custom `T` (e.g. a `bigint`-like type) that might not define it.

9.8 Summary table of constraints

Constraint	**Where enforced**
Allowed `T`	`static_assert` in `checkType()`, called from every `memvar<T>` constructor (runtime-of-instantiation, not class-declaration time)
Minimum history capacity (2)	Runtime `throw std::invalid_argument`, only in `memvar(const T&, capacityType)`
No `++`/`-`/`-=`/`*=`/`/=` for strings	`requires (!AnyStandardString<T>)` on the relevant members/free operators
No copy/move construction	`= delete` in both `memvarBase` and `memvar`/`memvarTimed`
No move assignment	`= delete` in both `memvarBase` and `memvar`/`memvarTimed`
Copy assignment allowed	User-defined in `memvar`/`memvarTimed`; copies only the current value, not history or capacity
Index bounds	`std::out_of_range` thrown by `getHistoryValue`/`deque::at`

Chapter 10

Conclusion

Taken together, `is_string.h` and `memvar.h` implement a compact, self-contained, header-only C++20 facility for scalar/string variables that retain a bounded history of their own past values, with an optional time-tagged variant. The design leans on modern C++20 facilities (concepts, `requires` clauses, `std::remove_cvref_t`, `std::ranges::minmax_element`) to selectively enable or disable operators per value category (numeric vs. string), and on a `std::deque`-based storage model that gives O(1) push-front/pop-back behavior appropriate for a bounded, newest-first history. The public API is broad — covering conversion, indexing, comparison, arithmetic, increment/decrement, printing, and reduction — but, as this analysis has shown from the code alone, it carries a number of rough edges: an unused error-flag field baked into the tuple types, formatting logic that only behaves correctly for a narrow default case (single-character separators on interactive terminals), an incompletely propagated capacity validation, a real desynchronization risk between `memvar`'s value history and `memvarTimed`'s time-tag history under exceptions, inconsistent signed/unsigned indexing conventions between `memvar` and `memvarTimed`, and free-function/operator conveniences for `memvarTimed` that silently stop working once its `Time`/`Clock` template parameters are changed from their defaults. None of these are fatal to the library's core use case, but they are the concrete points a maintainer or adopter should be aware of before relying on the library in the wider or more adversarial conditions this analysis has enumerated.